

Constructor University Bremen

Bachelor Thesis in Electrical and Computer Engineering

Spring Semester 2024

Generating Parity Check Matrices based on Degree Distributions

Written by: Abhigyan Deep Barnwal

Matriculation number: 30005327

Supervisor: Dr. Fangning Hu

Statutory Declaration

I hereby declare that the thesis submitted was created and written solely by myself without any external support. Any sources, direct or indirect, are marked as such. I am aware of the fact that the contents of the thesis in digital form may be revised with regard to the usage of unauthorized aid as well as whether the whole or parts of it may be identified as plagiarism. I do agree my work to be entered into a database for it to be compared with existing sources, where it will remain in order to enable further comparisons with future theses. This does not grant any rights of reproduction and usage, however.

This document was neither presented to any other examination board nor has it been published.

Date of Submission

Abhigyan Deep Barnwal
Full Name

.....
Signature

Abstract

Parity check (H) matrices for LDPC codes are randomly generated and sometimes have inefficiencies. Is there a way to generate a more efficient H-matrix?

In the modern age, digital communication requires increasingly stable connections and accurate, reliable data transfers. As the frequency and complexity of these transfers grow, the development of advanced error detection and correction algorithms becomes essential. Low-Density Parity-Check (LDPC) codes are a prevalent type of error-correcting code used to enhance data reliability. This thesis explores potential improvements to basic LDPC codes by examining the degrees of check nodes and variable nodes. Drawing inspiration from the paper "Irregular Repeat-Accumulate Codes," which posits that a specific fractional distribution of edges provides robust encoding, this study evaluates the effectiveness of these distributions. The results aim to contribute to the enhancement of LDPC codes, thereby supporting more reliable digital communication systems.

Contents

Abstract	2
Chapter 1 - Introduction	4
1.1 - Motivation and Objective	4
1.2 - Structure of the Thesis	5
Chapter 2 - Prerequisites	6
2.1 - Background Information	6
2.2 - Programming in Python	10
2.3 - Related Work	11
Chapter 3 - Methodology	12
3.1 - Generating the parity-check matrix	12
3.2 - Applying belief propagation using the H-matrix	18
Chapter 4 - Results and Analysis	27
Chapter 5 - Conclusion	30
Bibliography	31

Chapter 1 - Introduction

1.1 - Motivation and Objective

In the modern age, digital communication is one of the most critical forms of communication, necessitating more stable connections and accurate, reliable data transfers. As the frequency and complexity of data transfers increase, the need to develop more effective error detection and correction algorithms becomes paramount to ensure reliability.

Low-Density Parity-Check (LDPC) codes are a widely used type of error-correcting code in practice. Enhancing basic LDPC codes could significantly contribute to the reliability of data transfers. One potential area of improvement involves examining the degrees of check nodes and variable nodes within the LDPC codes.

The paper titled "Irregular Repeat–Accumulate Codes" suggests that a particular fractional distribution of edges results in robust encoding. This is particularly significant for 5G communication, where these edge distributions are anticipated to be highly effective. Motivated by these findings, these proposed distributions were tested to assess their performance and potential improvements in LDPC codes.

1.2 - Structure of the Thesis

Chapter 2 - Prerequisites

This chapter provides an overview of the existing theory and methods relevant to this thesis. It begins with a summary of the basic concepts of coding theory and LDPC codes. Following this, it discusses the construction of LDPC codes and the algorithms for both encoding and decoding these codes.

Chapter 3 - Methodology

This chapter will detail the proposed methods and algorithms, including the generation and utilization of the parity-check matrix H .

Chapter 4 - Simulation Results and Analysis

In this chapter, the methods provided in chapter 3 are applied and simulated in Python. The results show the effect of generating a parity check matrix based on the provided degree distributions.

Chapter 5 - Conclusion

The chapter concludes with a summary of the contributions made by this thesis and their significance in advancing the understanding and application of LDPC codes and their decoding algorithms.

Chapter 2 - Prerequisites

2.1 - Background Information

Error Correcting Codes

Error-correcting codes (ECCs) play a crucial role in communication systems by enabling reliable transmission of information over noisy channels. In essence, ECCs are methods for adding redundancy to data before transmission, allowing the receiver to detect and correct errors that occur during the communication process. This redundancy is achieved by encoding the original data into a codeword with additional bits, known as parity bits or check bits, which are calculated based on the original data.

The importance of ECCs is in their ability to reduce errors that inevitably occur in real-world communication channels due to noise, interference, or other sources of distortion. Without error correction, even small amounts of noise can corrupt the transmitted data, leading to the loss of information. This is especially critical in applications such as telecommunications, data storage, satellite communication, and wireless networks.

ECCs come in various forms, ranging from simple parity-check codes to more sophisticated block codes like Reed-Solomon and convolutional codes. Each type of ECC offers different trade-offs in terms of error-correction capability, complexity, and efficiency.

One of the key concepts in ECCs is the ability to balance redundancy and efficiency. While adding more redundancy improves error-correction capability, it also increases the overhead in terms of bandwidth or storage required. Therefore, ECC design involves optimizing the trade-off between error correction performance and resource utilization to meet the specific requirements of the communication system.

Overall, error-correcting codes are essential components of modern communication systems, enabling reliable transmission of data across noisy channels and ensuring seamless connectivity in a wide range of applications. Their importance continues to grow as communication technologies advance and the demand for high-speed, error-free data transmission increases.

This report focuses on a specific type of ECC known as Low-Density Parity Check (LDPC) codes, which are a class of error-correcting codes known for their excellent performance and efficiency.

LDPC Codes

LDPC (Low-Density Parity-Check) codes are a class of error-correcting codes characterized by a sparse parity-check matrix. In LDPC codes, this matrix consists mostly of zeros, leading to a low-density structure where only a small fraction of elements are non-zero. This sparsity is a key distinguishing feature of LDPC codes and facilitates efficient decoding algorithms.

LDPC codes are classified into two categories, regular and irregular. Regular LDPC codes have a uniform structure where each row and column of the parity-check matrix have the same weight, meaning they contain the same number of non-zero elements. On the other hand, irregular LDPC codes allow for variations in the number of non-zero elements per row and column, resulting in a more flexible code structure. Irregular LDPC codes can offer improved performance compared to regular LDPC codes, especially in terms of error-correction capability.

LDPC codes offer several advantages over other types of error-correcting codes. Firstly, their sparse parity-check matrix enables efficient decoding algorithms, such as the sum-product algorithm or belief propagation algorithm. These algorithms iteratively exchange messages between variable nodes (information bits) and check nodes (parity check bits), converging to the most likely transmitted codeword.

Another advantage of LDPC codes is their low error floor, which refers to the tendency to achieve near-optimal performance even at low signal-to-noise ratios (SNRs). This characteristic is crucial for communication systems operating in challenging environments where noise levels are high.

Furthermore, LDPC codes can approach the Shannon limit, which represents the theoretical maximum data rate for a given channel and noise level, making them highly attractive for applications requiring high-speed and reliable communication.

Belief Propagation Algorithm

Belief propagation is a message-passing algorithm used for decoding various types of graphical models, including error-correcting codes like LDPC codes. In the context of LDPC decoding, belief propagation operates by iteratively exchanging messages between variable nodes (information bits) and check nodes (parity check bits).

The algorithm starts by initializing messages with certain probabilities based on the received noisy signal. Then, in each iteration, variable nodes update their beliefs about the transmitted bits based on messages received from connected check nodes, and vice versa. This exchange of messages continues iteratively until either a valid codeword is found or a maximum number of iterations is reached.

Belief propagation leverages the concept of "belief" or probability distributions over possible values of variables, enabling efficient decoding by iteratively refining the estimates of transmitted bits. It's a powerful and widely used decoding algorithm due to its effectiveness and relatively low computational complexity.

Tanner Graphs

Tanner graphs are graphical representations used in coding theory, particularly for analyzing and decoding error-correcting codes like LDPC and turbo codes. Named after Michael Tanner, these graphs consist of two types of nodes: variable nodes representing the encoded bits and check nodes representing the parity checks imposed by the code. The relationship between encoded bits and parity checks is shown by edges that connect variable nodes to check nodes. Tanner graphs provide a visual depiction of the code structure, aiding in understanding and analyzing the decoding process, especially with iterative decoding algorithms like belief propagation.

Irregular Repeated Accumulator Codes

Accumulator codes are a type of error-correcting code that enhances reliability by repeating each bit multiple times. The repetition factor remains constant across all bits in a codeword, creating redundancy to detect and correct errors. Accumulator codes are simple to implement and offer a straightforward decoding process. They leverage iterative decoding algorithms, such as belief propagation, to refine estimates of transmitted bits based on the received information.

Irregular repeated accumulator (IRA) codes are a type of error-correcting code designed to improve upon traditional accumulator codes. In IRA codes, the repetition factor varies irregularly across bits, providing flexibility in balancing redundancy and error-correction capability. This irregularity allows for more efficient use of redundancy, enhancing the overall performance of the code. Unlike regular repetition codes, IRA codes exhibit irregular repetition patterns, depicted in their Tanner graph's connectivity. This irregularity allows for efficient use of redundancy while balancing error-correction capability. By leveraging Tanner graphs, IRA codes facilitate a visual understanding of the code's structure and aid in the development of efficient decoding algorithms, such as belief propagation. Consequently, IRA codes offer improved reliability and error-correction performance, making them valuable in various communication systems.

2.2 - Programming in Python

NumPy Library

In the realm of modern data analysis and numerical computing, the efficient handling of arrays and mathematical operations is essential. NumPy is a fundamental Python library that provides support for large, multi-dimensional arrays and matrices, along with a wide range of mathematical functions to operate on these arrays. It is widely recognized for its performance and versatility in scientific computing, data analysis, and engineering applications. NumPy's powerful array operations, including element-wise calculations and linear algebra routines, enable high-performance numerical computations and seamless integration with other scientific libraries. Its efficient handling of large datasets and mathematical operations makes it a cornerstone in the Python data science ecosystem.

H5PY Library

In the context of modern digital communication and data analysis, efficient storage and handling of large datasets are crucial. h5py is a powerful Python library that facilitates the use of the HDF5 (Hierarchical Data Format version 5) file format, which is designed for the management of large and complex data. HDF5 is widely used in various fields, including scientific research, engineering, and data science, due to its ability to store vast amounts of data in a flexible and efficient manner. The h5py library provides a simple and intuitive interface for reading and writing HDF5 files, allowing for integration of high-performance data management into Python applications.

2.3 - Related Work

The study of error-correcting codes is fundamental to improving the reliability of digital communication systems. LDPC codes, first introduced by Gallager in 1962, have become one of the most powerful error-correcting codes due to their capacity to approach the Shannon limit for large block lengths. Over the years, significant research has been conducted to enhance the performance of LDPC codes.

One notable advancement in the field is the development of irregular LDPC codes. Luby et al. demonstrated that irregular LDPC codes, which employ a non-uniform distribution of node degrees, can achieve better performance compared to regular LDPC codes. This irregularity allows for more flexible code design, enabling improved error correction capabilities.

The paper "Irregular Repeat–Accumulate Codes" by Divsalar et al. investigates the advantages of irregularity in LDPC codes. It suggests that a specific fractional distribution of edges leads to robust encoding, which can be particularly beneficial for modern communication systems such as 5G. The study proposes that optimizing the degrees of check nodes and variable nodes within the LDPC framework can enhance code performance.

Moreover, Richardson and Urbanke provided a detailed analysis of the design of LDPC codes using density evolution. Their work underscores the importance of node degree distribution in determining the threshold and error-floor performance of LDPC codes. They showed that carefully crafted degree distributions could significantly lower the error floor, making LDPC codes more effective for practical applications.

In the context of 5G communication, research by Arikan on polar codes introduced new possibilities for error correction, though LDPC codes remain highly relevant due to their flexibility and performance. Recent studies have focused on adapting LDPC codes to the stringent requirements of 5G networks, emphasizing low latency and high reliability.

This thesis builds upon the foundational work of Gallager and subsequent advancements by investigating the specific edge distributions proposed by Divsalar et al. The goal is to evaluate these distributions' performance in the context of 5G communication.

Chapter 3 - Methodology

3.1 - Generating the parity-check matrix

a	2	3	4
λ_2	0.139025	0.078194	0.054485
λ_3	0.222155	0.128085	0.104315
λ_5		0.160813	
λ_6	0.638820	0.036178	0.126755
λ_{10}			0.229816
λ_{11}			0.016484
λ_{12}		0.108828	
λ_{13}		0.487902	
λ_{14}			
λ_{16}			
λ_{27}			0.450302
λ_{28}			0.017842
rate	0.333364	0.333223	0.333218

Figure - Degree distributions from IRA paper

3.1.1 - Analyzing degree distributions and determining the number of information nodes

Given the degree distributions, we see that the check nodes must have a fixed left degree of 2, 3 or 4, meaning that each check node would have exactly a information bits connected to them. As such, the number of information bits that can be transmitted can be calculated by taking an arbitrary number of check nodes (1000) as a constant.

The tables below show the calculations done for each of the 3 cases.

Degree	Fractions of edges	Number of edges coming from a node of said degree	Number of information nodes of said degree	Sum check	Sum check
2	0.139025	278	139	278	2000
3	0.222155	444	148	444	
6	0.638820	1278	213	1278	
	1				
	a	no. of check nodes	no. of information nodes		
	2	1000	500		
		rate			
		0.333333			

As seen in the table above, a formula is used to compute the number of information nodes needed based on the check node. First, the number of check nodes (1000) is multiplied by the constant left degree of a check node to compute the total number of edges between the information nodes and the check nodes (2000 in this case). Next, that value is multiplied by each of the fractions to compute how many of the edges come from information nodes of the given degrees. These numbers are then divided by the degree of the respective node to compute how many information node of each degree there will be, and then the sum is calculated to compute the total number of edges and ensure that it is equal to the original total that it should be (which is 2000 in this case). The reason for the sum checking is more apparent in the following table showing the calculations for the case with $\alpha = 3$.

Degree	a=3	Number of edges coming from a node of said degree	Number of information nodes of said degree	Sum check	Sum check	Degree	Corrected number of information nodes of said degree	Correct ed sum check	Correct ed sum check
2	0.078194	235	117	234	2999	2	116	232	3000
3	0.128085	384	128	384		3	129	387	
5	0.160813	482	96	480		5	96	480	
6	0.036178	109	18	108		6	18	108	
12	0.108828	326	27	324		12	27	324	
13	0.487902	1464	113	1469		13	113	1469	
	1	3000							
		no. of check a nodes	no. of information nodes						
	3	1000	499						
		rate							
		0.332889							

As seen in the table above, after the sum check, the calculation shows 499 information nodes, but 2999 edges, which is not the expected 3000. The reason for this happening is simply due to rounding errors. As one more edge is needed to account for this, one of the information nodes with degree 2 has to be changed to 3. Below is the corresponding computation for $a = 4$.

Degree	a=4	number of edges coming from a node of said degree	number of information nodes of said degree	sum check	sum check	Degree	corrected number of information nodes of said degree	Correct ed sum check	Correcte d sum check
2	0.054485	218	109	218	4024	2	109	218	4000
3	0.104315	417	139	417		3	140	420	
6	0.126755	507	85	510		6	85	510	
10	0.229816	919	92	920		10	92	920	
11	0.016484	66	6	66		11	6	66	
27	0.450302	1801	67	1809		27	66	1782	
28	0.017842	71	3	84		28	3	84	
	1	4000							
		no. of check a nodes	no. of variable nodes						
	4	1000	501						
		rate							
		0.333777							

As seen in the table above, as the numbers get larger, so does the rounding error. In this case, the sum of edges exceeded the total, and as the error difference was exactly the same as the difference between 27 and 3, one of the information nodes with 27 edges was reduced to 3 to balance it out. Also, to check the formulas, the tables were tested for higher numbers of check nodes to ensure the code rate matched the one given in the paper.

a	no. of check nodes	no. of information nodes	rate
2	10000000	5000683	0.333364
3	10000000	4997526	0.333223
4	10000000	4997409	0.333218

As can be seen, the code rate value, calculated by dividing the number of information nodes by the total number of nodes, matches the values given in the paper. However, due to computing restrictions, using 10000000 check nodes is impractical, hence 1000 was used for the number of check nodes.

3.1.2 - Generating the parity check matrix based on the degree distributions

Using the now calculated number of information nodes for each degree, the H-matrix can now be programmed. Below is the code to compute the left degrees of the check nodes.

```
import numpy as np
import random
import h5py

def generate_parity_check_matrix_left(num_variable_nodes, num_check_nodes,
degree_distribution):
    # Accounting for the degree distribution
    degree_2_count, degree_3_count, degree_6_count = degree_distribution
    degrees = [2] * degree_2_count + [3] * degree_3_count + [6] * degree_6_count

    # Checking if sum of degrees matches twice the number of check nodes
    assert sum(degrees) == 2 * num_check_nodes, "The degree distribution does not match the
constraints."

    # Initialize the H matrix with zeros
    H = np.zeros((num_check_nodes, num_variable_nodes), dtype=int)

    # List of all edges
    edges = [(i, j) for i in range(num_check_nodes) for j in range(2)]

    # Assign edges based on the degree distribution
    variable_node_edges = []
    for node, degree in enumerate(degrees):
        variable_node_edges.extend([node] * degree)

    random.shuffle(variable_node_edges)

    # Now assign edges from variable_node_edges to check nodes
    check_node_index = 0
    while variable_node_edges:
        if len(edges) < 2:
            raise ValueError("Not enough edges to fulfill the degree distribution.")

        v_node = variable_node_edges.pop()
```

```

# Assign the edge to a check node ensuring each check node gets exactly 2 edges
while True:
    if check_node_index >= num_check_nodes:
        check_node_index = 0
    c_node = check_node_index
    check_node_index += 1
    if sum(H[c_node]) < 2:
        H[c_node, v_node] = 1
        break

return H

# Function to save H matrix in h5py file
def save_h_matrix_to_hdf5(H, filename):
    with h5py.File(filename, "w") as f:
        f.create_dataset("H2", data=H)

def main():

    # Setting parameters as calculated
    num_variable_nodes = 500
    num_check_nodes = 1000
    degree_distribution = (139, 148, 213)

    # Generating the H-matrix and saving it to hdf5 file
    H_left = generate_parity_check_matrix_left(num_variable_nodes, num_check_nodes,
    degree_distribution)
    save_h_matrix_to_hdf5(H_left, "H_matrix_2.hdf5")
    print("Matrix was successfully saved to hdf5 file")

if __name__ == "__main__":
    main()

```

The matrix is saved to an hdf5 file format to be able to view it and check whether the matrix is 'correct' or not. However, the code above only gives the H-matrix for the left connections of the check nodes, not the right connections, which are to the parity nodes. The right part is then computed and combined with the left part as follows.

```

def generate_parity_check_matrix(num_variable_nodes, num_check_nodes,

```

```

degree_distribution):

    H_left = generate_parity_check_matrix_left(num_variable_nodes, num_check_nodes,
degree_distribution)

    H_right = np.zeros((num_check_nodes, num_check_nodes), dtype=int)
    for i in range(num_check_nodes):
        H_right[i, i] = 1
        if(i+1)<num_check_nodes:
            H_right[i, i+1] = 1

    H_real = np.zeros((num_check_nodes, num_variable_nodes + num_check_nodes),
dtype=int)
    H_real[:, :num_variable_nodes] = H_left
    H_real[:, num_variable_nodes:] = H_right

    return H_real, H_left

```

This results in the full H-matrix that has each row representing a check node's connections to information and parity nodes, collectively referred to as variable nodes, and each column representing a given variable node's connection to check nodes.

3.2 - Applying belief propagation using the H-matrix

After generating the full matrix, to simulate the full LDPC code process, an initial signal is generated. In this case, as tests are being done, the message signal is just 0s. The signal is then multiplied element-wise to the left part of the H-matrix and summed mod 2 to get the check nodes. Next the check nodes are added mod 2 are per their connections to the parity nodes, to get the values of the parity nodes. The message signal and the parity nodes are then concatenated to get the full transmitted signal. The code is shown below.

```

def get_transmitted_signal(H_left, initial_signal, num_check_nodes):
    # Element-wise multiplication of the vector with every row of the matrix
    redundancy_temp = H_left * initial_signal

    # Sum up each row

```

```

redundancy_temp2 = np.sum(redundancy_temp, axis=1) % 2

# Calculating the parity nodes
redundancy = np.zeros((num_check_nodes), dtype=int)

for i in range(len(redundancy_temp2) - 1):
    redundancy[i] = (redundancy_temp2[i] + redundancy_temp2[i+1]) % 2

redundancy[len(redundancy_temp2)-1] = redundancy_temp2[len(redundancy_temp2)-1]

# Compiling the full message signal
transmitted_signal = np.concatenate((initial_signal, redundancy))

return transmitted_signal

```

Average white gaussian noise is then added to the transmitted signal to simulate passing through free space.

```

def add_awgn(signal, snr_dB):
    snr_linear = 10**(snr_dB / 10.0)
    power_signal = np.mean(signal**2)
    noise_power = power_signal / snr_linear
    noise = np.sqrt(noise_power) * np.random.randn(*signal.shape)
    noisy_signal = signal + noise
    return noisy_signal, noise_power

def get_noisy_signal(transmitted_signal, snrdB):
    # Adding noise to the signal
    bpsk_signal = 2 * transmitted_signal - 1 # Convert to BPSK: 0 -> -1, 1 -> +1
    noisy_signal, noise_power = add_awgn(bpsk_signal, snrdB)

    return noisy_signal, noise_power

```

Once the signal is received, the Log Likelihood Ratios are calculated and belief propagation is used by implementing the message passing algorithm, and the results are plotted using the matplotlib library. The full code for $a = 2$ is given below:

```

import numpy as np
import random
import matplotlib.pyplot as plt

def generate_parity_check_matrix_left(num_variable_nodes, num_check_nodes,
degree_distribution):
    # Accounting for the degree distribution
    degree_2_count, degree_3_count, degree_6_count = degree_distribution
    degrees = [2] * degree_2_count + [3] * degree_3_count + [6] * degree_6_count

    # Checking if sum of degrees matches twice the number of check nodes
    assert sum(degrees) == 2 * num_check_nodes, "The degree distribution does not match the
constraints."

    # Initialize the H matrix with zeros
    H = np.zeros((num_check_nodes, num_variable_nodes), dtype=int)

    # List of all edges
    edges = [(i, j) for i in range(num_check_nodes) for j in range(2)]

    # Assign edges based on the degree distribution
    variable_node_edges = []
    for node, degree in enumerate(degrees):
        variable_node_edges.extend([node] * degree)

    random.shuffle(variable_node_edges)

    # Now assign edges from variable_node_edges to check nodes
    check_node_index = 0
    while variable_node_edges:
        if len(edges) < 2:
            raise ValueError("Not enough edges to fulfill the degree distribution.")

        v_node = variable_node_edges.pop()

        # Assign the edge to a check node ensuring each check node gets exactly 2 edges
        while True:
            if check_node_index >= num_check_nodes:
                check_node_index = 0
            c_node = check_node_index

```

```

        check_node_index += 1
        if sum(H[c_node]) < 2:
            H[c_node, v_node] = 1
            break

    return H

def generate_parity_check_matrix(num_variable_nodes, num_check_nodes,
degree_distribution):
    # Generate the parity check matrix
    H_left = generate_parity_check_matrix_left(num_variable_nodes, num_check_nodes,
degree_distribution)

    H_right = np.zeros((num_check_nodes, num_check_nodes), dtype=int)
    for i in range(num_check_nodes):
        H_right[i, i] = 1
        if (i+1) < num_check_nodes:
            H_right[i, i+1] = 1

    H_real = np.zeros((num_check_nodes, num_variable_nodes + num_check_nodes),
dtype=int)
    H_real[:, :num_variable_nodes] = H_left
    H_real[:, num_variable_nodes:] = H_right

    return H_real, H_left

def add_awgn(signal, snr_dB):
    snr_linear = 10**((snr_dB / 10.0))
    power_signal = np.mean(signal**2)
    noise_power = power_signal / snr_linear
    noise = np.sqrt(noise_power) * np.random.randn(*signal.shape)
    noisy_signal = signal + noise
    return noisy_signal, noise_power

def get_llr_from_initial_signal(H_left, initial_signal, snr_dB, num_check_nodes):
    # Element-wise multiplication of the vector with every row of the matrix
    redundancy_temp = H_left * initial_signal

    # Sum up each row
    redundancy_temp2 = np.sum(redundancy_temp, axis=1) % 2

```

```

# Calculating the parity nodes
redundancy = np.zeros((num_check_nodes), dtype=int)

for i in range(len(redundancy_temp2) - 1):
    redundancy[i] = (redundancy_temp2[i] + redundancy_temp2[i+1]) % 2

redundancy[len(redundancy_temp2)-1] = redundancy_temp2[len(redundancy_temp2)-1]

# Compiling the full message signal
generated_signal = np.concatenate((initial_signal, redundancy))

# Adding noise to the signal
bpsk_signal = 2 * generated_signal - 1 # Convert to BPSK: 0 -> -1, 1 -> +1
noisy_signal, noise_power = add_awgn(bpsk_signal, snrdB)

# Calculate LLRs
llr = (2 * noisy_signal) / noise_power

return llr

def belief_propagation(H_real, received_llr, max_iterations):

    eps=1e-6
    M, N = H_real.shape
    # Initialize messages

    r = np.zeros((M, N)) # messages from check nodes to variable nodes
    q = np.zeros((M, N)) # messages from variable nodes to check nodes

    # Initialize q messages with received LLRs
    for m in range(M):
        for n in np.where(H_real[m])[0]:
            q[m, n] = received_llr[n]

    for iteration in range(max_iterations):
        # Messages sent by check node to variable node
        for m in range(M):
            for n in np.where(H_real[m])[0]:

```

```

    neighbors = np.where(H_real[m])[0]
    neighbors = neighbors[neighbors != n]
    prod = np.prod(np.tanh(q[m, neighbors] / 2))
    prod = np.clip(prod, -1 + eps, 1 - eps)
    r[m, n] = 2 * np.arctanh(prod)

# Messages sent by variable node to check node
llr_updated = np.copy(received_llr)
for n in range(N): # for each variable node
    for m in np.where(H_real[:, n])[0]: # for each check node connected to it
        neighbors = np.where(H_real[:, n])[0]
        neighbors = neighbors[neighbors != m]
        sum = np.sum(r[neighbors, n]) + received_llr[n]
        q[m, n] = sum
        llr_updated[n] += sum

# Decision
decoded_bits = (llr_updated > 0).astype(int)
# decoded_bits = llr_updated

# Check if all parity-check equations are satisfied
syndrome = np.mod(H_real @ decoded_bits, 2)
# print(np.sum(syndrome))
if np.all(syndrome == 0):
    return decoded_bits, iteration + 1

# Update q messages for next iteration
for m in range(M):
    for n in np.where(H_real[m])[0]:
        neighbors = np.where(H_real[:, n])[0]
        neighbors = neighbors[neighbors != m]
        q[m, n] = received_llr[n] + r[neighbors, n].sum()

return decoded_bits, max_iterations

# calculate ber
def calculate_ber(original_bits, decoded_bits):
    # check if both inputs are the right dimensions
    if original_bits.shape != decoded_bits.shape:
        raise ValueError("Length of original_bits and decoded_bits must be the same.")

```



```

# calculate and return ber
return np.sum(original_bits != decoded_bits) / len(original_bits)

def simulate_ldpc(H, H_left, initial_signal, snr_values, max_iter, num_check_nodes):
    ber = []
    avg_iterations = []

    for snr_dB in snr_values:
        llr = get_llr_from_initial_signal(H_left, initial_signal, snr_dB, num_check_nodes)

        # Decode the received signal
        decoded_bits, iterations = belief_propagation(H, llr, max_iter)

        # Getting the message signal from the decoded bits
        decoded_message = decoded_bits[:len(initial_signal)]

        # Calculate BER and store results
        ber_snr = calculate_ber(initial_signal, decoded_message)
        ber.append(ber_snr)
        avg_iterations.append(iterations)

        # debugging
        #print(f'SNR: {snr_dB}")
        #print(f'Number of Iterations: {iterations}")
        #print(f'Bit Error Rate (BER): {ber_snr}")

    return ber, avg_iterations

def plot_results(snr_values, ber, avg_iterations):
    plt.figure(figsize=(12, 6))

    # BER vs SNR plot
    plt.subplot(1, 2, 1)
    plt.plot(snr_values, ber, marker='o')
    plt.title('BER vs SNR')
    plt.xlabel('SNR (dB)')
    plt.ylabel('Bit Error Rate (BER)')
    plt.grid(True, which='both')
    plt.xlim(-4, 11)
    plt.ylim(0, 0.25)

```

```

# Average Iterations vs SNR plot
plt.subplot(1, 2, 2)
plt.plot(snr_values, avg_iterations, marker='o')
plt.title('Average Iterations vs SNR')
plt.xlabel('SNR (dB)')
plt.ylabel('Average Iterations')
plt.grid(True)
plt.xlim(-4, 11)
plt.ylim(0, 50)

plt.tight_layout()
plt.show()

def main():

    # Setting parameters
    num_variable_nodes = 500
    num_check_nodes = 1000
    degree_distribution = (139, 148, 213)
    snr_values = np.arange(-3, 11, 1)
    max_iter = 50

    # Generating parity check matrix
    H_real, H_left = generate_parity_check_matrix(num_variable_nodes, num_check_nodes,
    degree_distribution)

    # Generating initial signal
    initial_signal = np.random.randint(0, 2, num_variable_nodes)

    # Performing the encoding, transmission and decoding process
    ber, avg_iterations = simulate_ldpc(H_real, H_left, initial_signal, snr_values, max_iter,
    num_check_nodes)

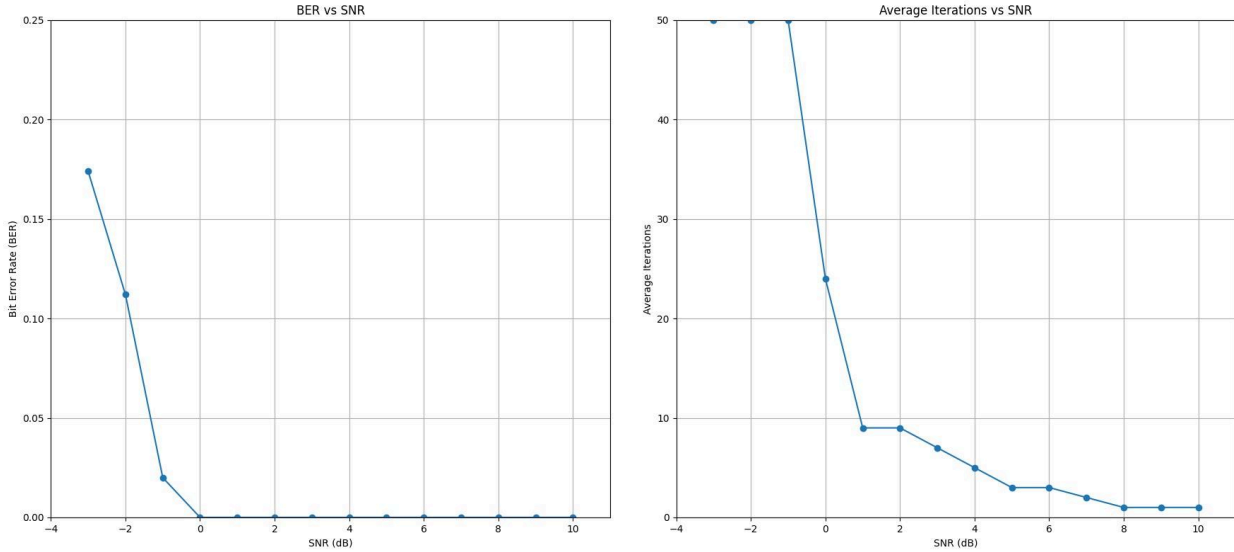
    # Plotting the results
    plot_results(snr_values, ber, avg_iterations)

if __name__ == "__main__":
    main()

```

Chapter 4 - Results and Analysis

The output of the given code is shown below:



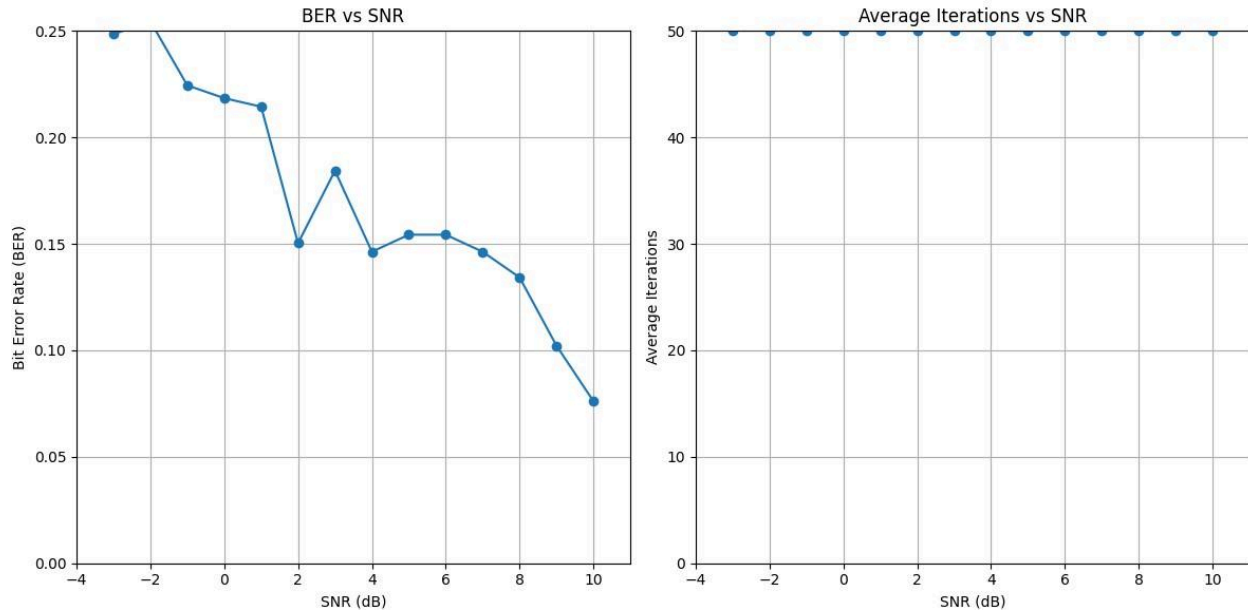
Results for $a = 2$

The BER (Bit Error Rate) decreases rapidly as the SNR (Signal-to-Noise Ratio) increases.

Initially, at low SNR values, the BER starts around 0.2, indicating a high error rate. However, as the SNR increases, the BER drops significantly, reaching almost 0 around an SNR of 0 dB. This means that at higher SNR levels, the original message is fully recovered with minimal errors.

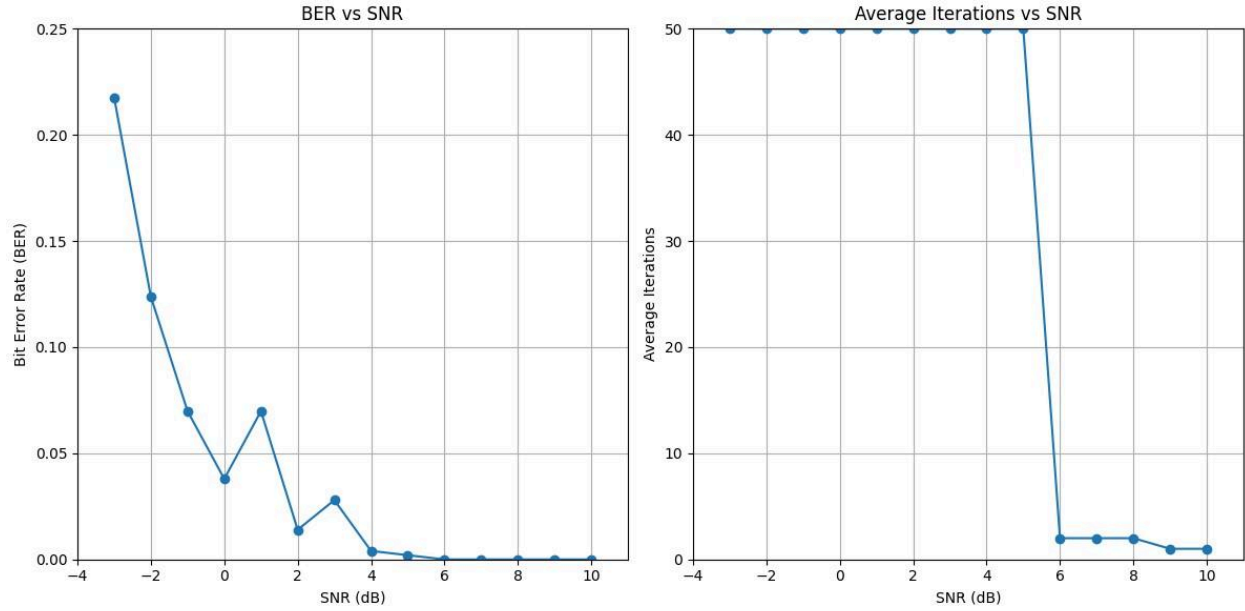
The number of iterations required for decoding decreases sharply with increasing SNR. Initially, the system requires about 50 iterations to decode the message, but as the SNR improves, the iterations needed drop significantly, converging to very few iterations around an SNR of 2 dB.

This indicates that the code is efficient and converges quickly, leading to effective error correction at higher SNR levels.



Results for $a = 3$

Unlike the previous case, the BER does not decrease as quickly for $a = 3$. Even at moderate SNR levels, the BER remains relatively high, fluctuating between 0.1 and 0.15. This indicates that the code struggles to fully recover the signal, even as the SNR improves. At higher SNR values, while there is some reduction in BER, it remains higher compared to the case with $a = 2$. The number of iterations remains consistently high across the SNR spectrum, showing minimal decrease even at higher SNRs. This suggests that the code with $a = 3$ is less efficient, requiring more iterations to attempt decoding, and yet still fails to fully correct errors, as indicated by the higher BER.



Results for $a = 4$

The performance with $a = 4$ mirrors that of $a = 2$, with the BER decreasing rapidly as SNR increases. The BER drops to nearly 0 around 0 dB SNR, and this low error rate is maintained as the SNR increases further. This demonstrates that the code effectively recovers the original signal at higher SNRs. Similar to the case of $a = 2$, the number of iterations required decreases significantly with increasing SNR. The system starts with about 50 iterations but quickly drops to minimal iterations as SNR improves, indicating fast convergence and efficient error correction.

From the results, it is evident that LDPC codes with $a = 2$ and $a = 4$ show robust performance in error correction, characterized by a sharp decline in BER as SNR increases, and a corresponding reduction in the number of iterations needed for decoding. This implies that these codes are highly effective in recovering the original message signal at higher SNR values, requiring fewer iterations and resulting in a low bit error rate.

On the other hand, the code with $a = 3$ underperforms in comparison. The BER remains relatively high even at improved SNR levels, and the number of iterations required does not decrease significantly. This suggests that the code is less capable of fully recovering the signal, leading to a higher residual error rate.

Overall, this analysis demonstrates that the LDPC codes with $a = 2$ and $a = 4$ are more robust and efficient in error correction than those with $a = 3$ under the tested conditions. This highlights the importance of choosing the appropriate degree distribution to optimize the performance of LDPC codes in practical applications.

Chapter 5 - Conclusion

Based on the results obtained, we can conclude that the proposed degree distributions for check nodes with constant left degrees of 2 and 4 demonstrate strong performance, effectively enhancing the reliability of LDPC codes in the context of the studied applications. These configurations showed a marked improvement in error correction capability and overall system performance, validating the efficacy of the specific edge distributions analyzed.

However, the check nodes with a constant left degree of 3 did not perform as well as expected. This discrepancy presents an interesting anomaly that warrants further investigation. The underlying reasons for the suboptimal performance at this degree remain unclear from the current study. Potential contributing factors could include the specific interaction between the node degree and the overall structure of the LDPC code, or perhaps the influence of other variables not fully accounted for in this analysis.

Future research could delve into these aspects more deeply, exploring the intricacies of how different left degree configurations affect code performance. This could involve a more detailed examination of the encoding and decoding processes, as well as simulations under a wider range of conditions. Such studies could provide a clearer understanding of the limitations observed with a left degree of 3 and offer insights that might lead to further optimization of LDPC codes.

Bibliography

- Abbasfar, Aliazam, Dariush Divsalar, and Kung Yao. "Accumulate-repeat-accumulate codes." *IEEE Transactions on Communications* 55.4 (2007): 692-702.
- Arikan, Erdal. "Channel Polarization: A Method for Constructing Capacity-Achieving Codes for Symmetric Binary-Input Memoryless Channels." *IEEE Transactions on Information Theory*, vol. 55, no. 7, 2009, pp. 3051-3073.
- Collette, Andrew. *Python and HDF5: unlocking scientific data*. "O'Reilly Media, Inc.", 2013.
- Divsalar, Dariush, et al. "Irregular Repeat–Accumulate Codes." *IEEE Transactions on Information Theory*, vol. 47, no. 2, 2001, pp. 1413-1430.
- Gallager, Robert G. *Low-Density Parity-Check Codes*. MIT Press, 1963.
- Harris, C. R., et al. "Array Programming with NumPy." *Nature*, vol. 585, no. 7825, 2020, pp. 357-362.
<https://doi.org/10.1038/s41586-020-2649-2>.
- Jiang, Jing, and Krishna R. Narayanan. "Iterative soft-input soft-output decoding of Reed–Solomon codes by adapting the parity-check matrix." *IEEE Transactions on Information Theory* 52.8 (2006): 3746-3756.
- Jin, Hui, et al. "Irregular Repeat Accumulate Codes." *Department of Electrical Engineering, California Institute of Technology Pasadena*, Jan. 2000.
- Luby, Michael G., et al. "Improved Low-Density Parity-Check Codes Using Irregular Graphs." *IEEE Transactions on Information Theory*, vol. 47, no. 2, 2001, pp. 585-598.
- Richardson, Tom, and Rüdiger Urbanke. "The Capacity of Low-Density Parity-Check Codes under Message-Passing Decoding." *IEEE Transactions on Information Theory*, vol. 47, no. 2, 2001, pp. 599-618.
- Ryan, William E. "An introduction to LDPC codes." *CRC Handbook for Coding and Signal Processing for Recording Systems* 5.2 (2004): 1-23.
- Simonosk, Oliver, et al. *Coding Schemes in 5G Networks for Error-Free Communication*,
dergipark.org.tr/en/download/article-file/1931757. Accessed 15 July 2024.